Звіт про виконання завдання № 5.5

Рекурсивні функції: системи числення, рівень і глибина рекурсії

Варіант 7

Виконав: Джамалов Олександр Олегович, група РІ-11

1. Мета роботи

Навчитися реалізовувати рекурсивні функції, визначати рівень і глибину рекурсії.

2. Умова завдання

Для всіх варіантів визначити та вивести глибину та рівень рекурсії.

Варіант 7.

Визначити рекурсивні функції зображення натурального числа у двійковій, вісімковій системах числення та у системі числення з основою $N > 1$ та глибину рекурсії. Числа у десятковій системі числення вводити з клавіатури.

Рисунок 1 – Умова завдання 5.5, варіант 7

3. Математична модель

Запис числа n в системі з основою N: старші цифри – це запис числа n div N, остання цифра – n mod N. Отже, рекурсивна функція викликає себе для n div N і дописує цифру n mod N після повернення (на підйомі). Базовий випадок: n < N – число складається з однієї цифри.

Рівень рекурсії – номер поточного виклику (перший виклик має рівень 1), він передається параметром level. Глибина рекурсії – найбільший досягнутий рівень: depth = max(depth, level), передається параметром-посиланням. Глибина дорівнює кількості цифр результату. Цифри 10…35 позначаються A…Z, цифри ≥ 36 – як [значення].

4. Структура програми

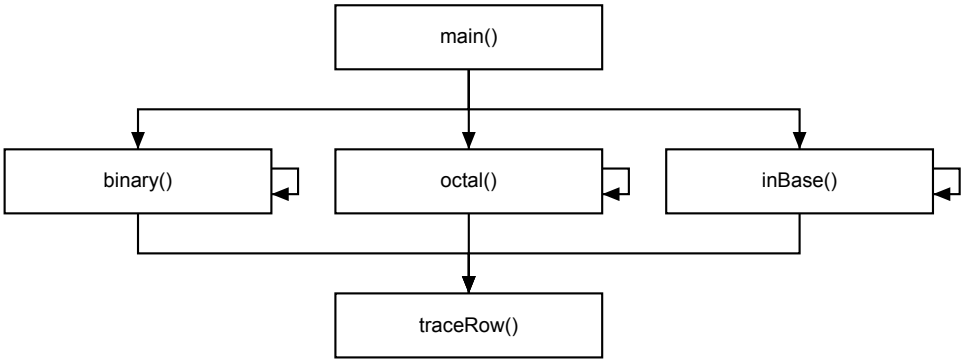


Рисунок 2 – Структурна схема програми (стрілка праворуч – рекурсивний виклик)

Таблиця 1 – Специфікація функцій програми

Функція	Призначення	Параметри	Результат
string binary(long long n, int level, int& depth, ostream& trace)	запис n у двійковій системі	n – число; level – рівень; depth – глибина; trace – потік	рядок цифр; depth
string octal(long long n, int level, int& depth, ostream& trace)	запис n у вісімковій системі	як у binary	рядок цифр; depth
string inBase(long long n, long long base, int level, int& depth, ostream& trace)	запис n у системі з основою N	base = N; інші – як у binary	рядок цифр; depth
void traceRow(ostream& trace, int level, long long n, const string& digit)	рядок таблиці трасування	level, n, digit	–
int main()	введення n, N; виклики; таблиці	–	код завершення

Таблиця 2 – Змінні програми

Ім'я	Тип	Функція	Призначення
n	long long	усі	число, що переводиться (у рекурсії – частка)
base	long long	main, inBase	основа системи числення N
level	int	binary, octal, inBase	рівень поточного виклику
depth / d2, d8, dN	int	рекурсивні / main	глибина рекурсії
digit	string	рекурсивні	остання цифра n mod N
r	long long	inBase	остача n mod N
r2, r8, rN	string	main	записи числа в системах 2, 8, N

5. Алгоритми програми

Блок-схеми виконано за ГОСТ 19.701-90 (ДСТУ ISO 5807:2016): «Початок» і «Кінець» – символ «термінатор», введення/виведення – «дані» (паралелограм), обчислення – «процес», виклик функції – «зумовлений процес», умова – «рішення». Лінії потоку, що йдуть зверху вниз і зліва направо, стрілками не позначено; стрілку показано лише на лінії повернення (повторення циклу). Рекурсивний виклик зображено символом «зумовлений процес».

UML-діаграми діяльності (UML 2.5) виконано в рамці «act»: початковий вузол – зафарбоване коло, кінцевий – коло з точкою, дії – прямокутники із заокругленими кутами (дія-виклик позначена символом «вила»), розгалуження та злиття – ромби, умови переходів – сторожові вирази у квадратних дужках. Усі переходи позначено стрілками.

5.1. Головна функція main()

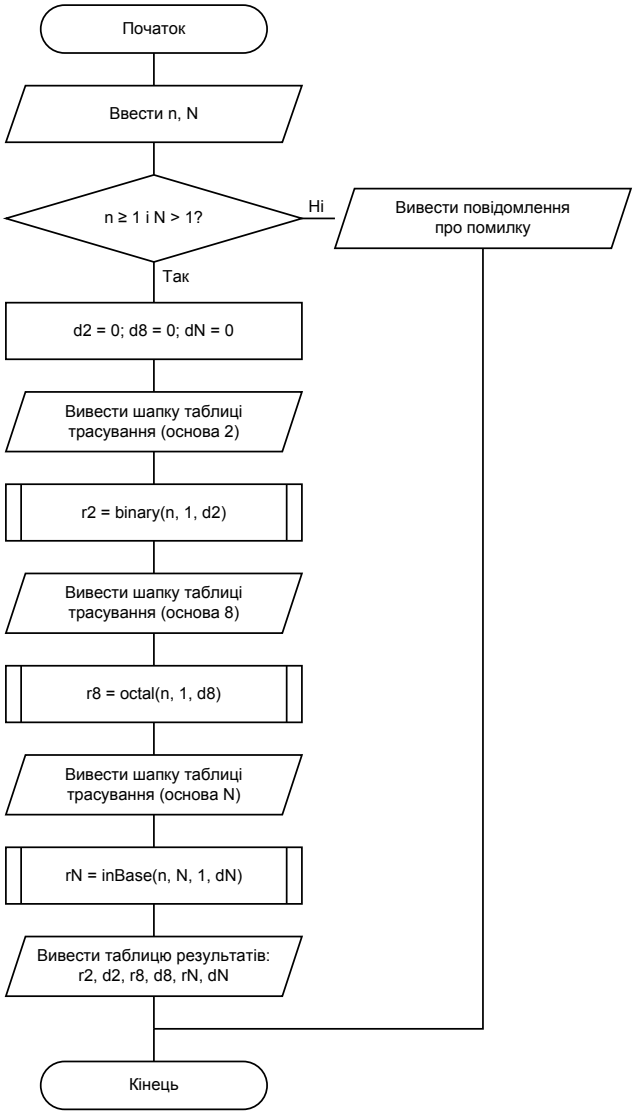


Рисунок 3 – Блок-схема функції main()

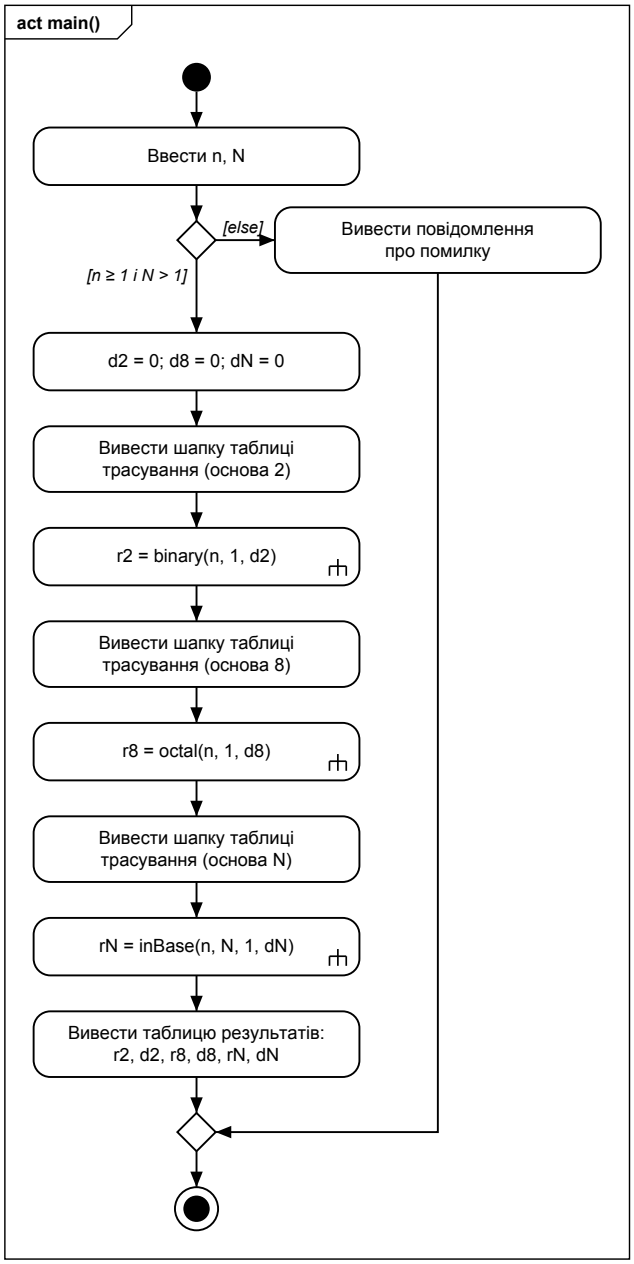


Рисунок 4 – UML-діаграма діяльності функції main()

5.2. Функція binary(n, level, depth, trace)

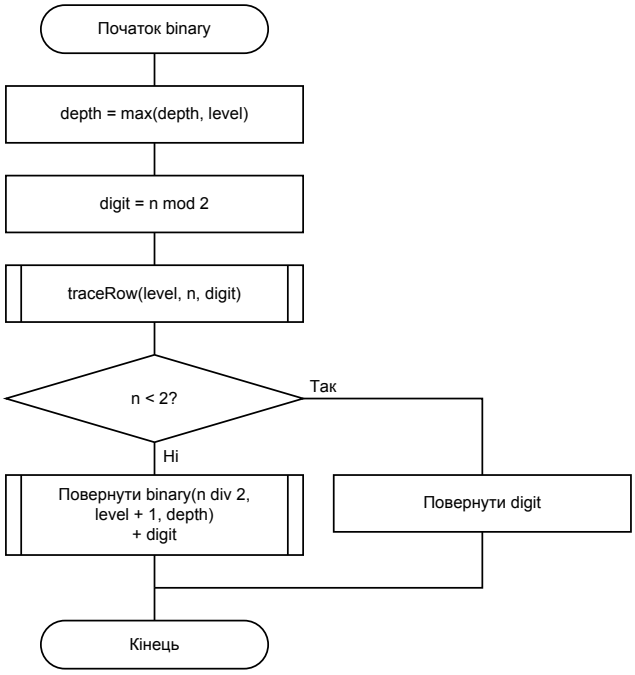


Рисунок 5 – Блок-схема функції binary

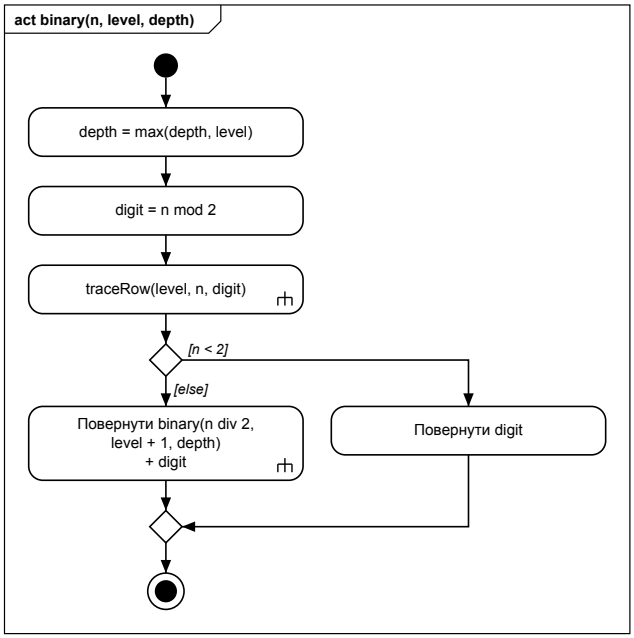


Рисунок 6 – UML-діаграма діяльності функції binary

5.3. Функція octal(n, level, depth, trace)

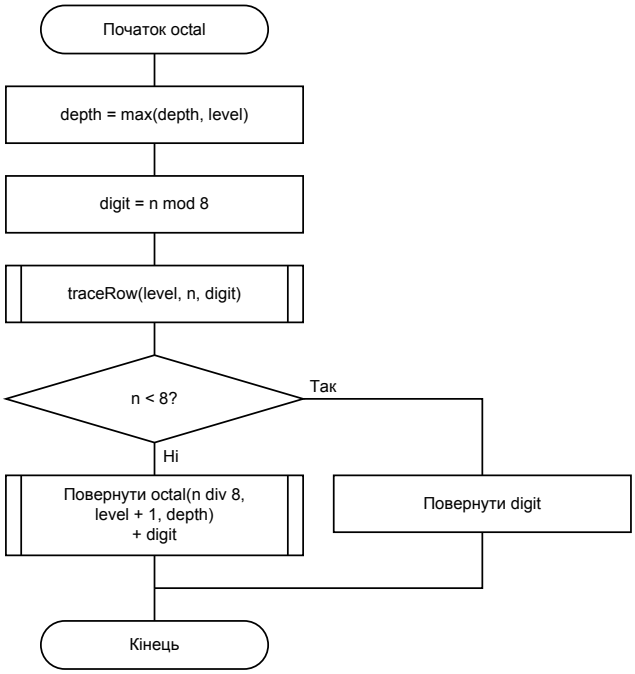


Рисунок 7 – Блок-схема функції octal

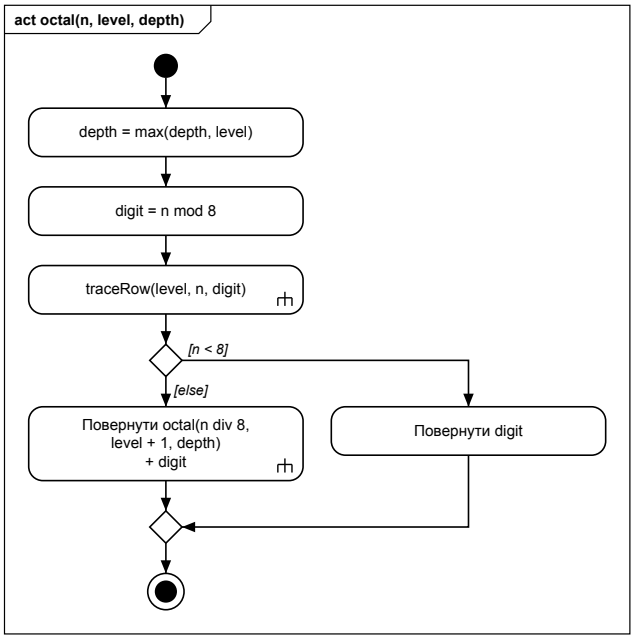


Рисунок 8 – UML-діаграма діяльності функції octal

5.4. Функція inBase(n, N, level, depth, trace)

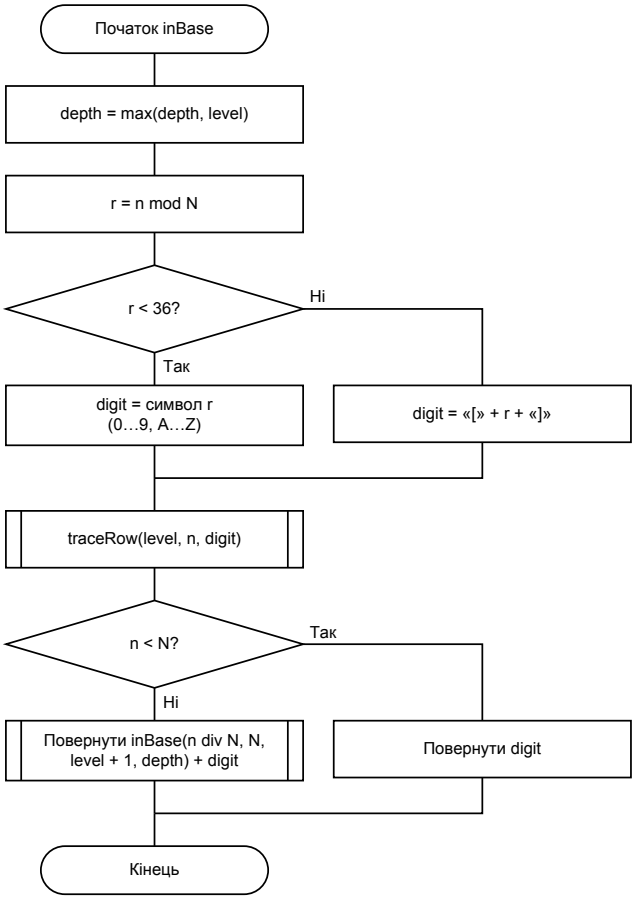


Рисунок 9 – Блок-схема функції inBase

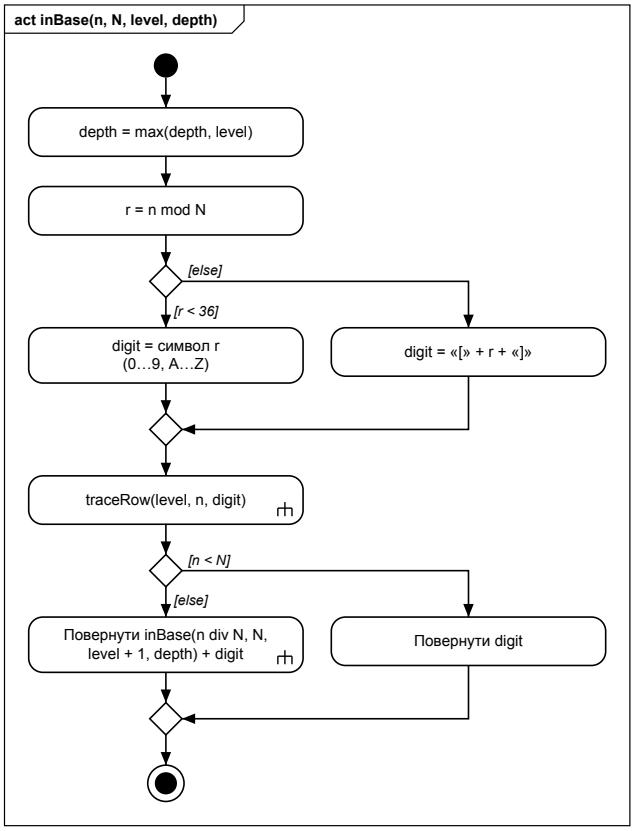


Рисунок 10 – UML-діаграма діяльності функції inBase

5.5. Функція traceRow(trace, level, n, digit)



Рисунок 11 – Блок-схема функції traceRow

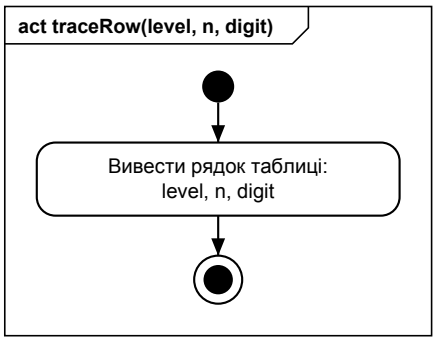


Рисунок 12 – UML-діаграма діяльності функції traceRow

6. Текст програми

Кожна дія програми супроводжується коментарем. Головну функцію main() вимкнено директивою #ifndef UNIT_TEST, щоб файл можна було підключити до тестів.

Файл `lab_5_5.cpp`

```
// Лабораторна робота 5.5, варіант 7. Джамалов О. О., PI-11.
// Рекурсивне зображення натурального числа в системах з основами 2, 8 і N > 1;
// виведення рівня кожного виклику та глибини рекурсії.
#include <algorithm> // max
#include <iomanip>   // setw
#include <iostream>  // cin, cout, cerr
#include <string>    // string, to_string
using namespace std;

// Рядок трасування: рівень рекурсії, поточне число, цифра (остача).
void traceRow(ostream& trace, int level, long long n, const string& digit)
{
    trace << "| " << setw(5) << level << " | " << setw(20) << n
          << " | " << setw(6) << digit << " |\n"; // один рядок на один виклик
}

// Двійкова система: старші цифри - рекурсивно, остання - n % 2.
string binary(long long n, int level, int& depth, ostream& trace)
{
    depth = max(depth, level);                  // оновлюємо глибину
    const string digit = to_string(n % 2);      // остання двійкова цифра
    traceRow(trace, level, n, digit);           // виводимо рівень і число
    if (n < 2)                                  // базовий випадок: одна цифра
        return digit;
    return binary(n / 2, level + 1, depth, trace) + digit; // рекурсія + цифра
}

// Вісімкова система: старші цифри - рекурсивно, остання - n % 8.
string octal(long long n, int level, int& depth, ostream& trace)
{
    depth = max(depth, level);                  // оновлюємо глибину
    const string digit = to_string(n % 8);      // остання вісімкова цифра
    traceRow(trace, level, n, digit);           // виводимо рівень і число
    if (n < 8)                                  // базовий випадок: одна цифра
        return digit;
    return octal(n / 8, level + 1, depth, trace) + digit; // рекурсія + цифра
}

// Довільна основа N > 1; цифри 10..35 - A..Z, цифри >= 36 - [значення].
string inBase(long long n, long long base, int level, int& depth, ostream& trace)
{
    depth = max(depth, level);                  // оновлюємо глибину
    const string digits = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"; // алфавіт цифр
    const long long r = n % base;               // остання цифра (остача)
    const string digit = r < 36 ? string(1, digits[r])
                                : "[" + to_string(r) + "]"; // запис цифри
    traceRow(trace, level, n, digit);           // виводимо рівень і число
    if (n < base)                               // базовий випадок: одна цифра
        return digit;
    return inBase(n / base, base, level + 1, depth, trace) + digit; // рекурсія + цифра
}

#ifndef UNIT_TEST
int main()
{
    long long n, base;                          // число та основа N
    cout << "n N (n >= 1, N > 1): ";            // запрошення до введення
    if (!(cin >> n >> base) || n < 1 || base < 2) // перевірка введення
    {
        cerr << "Error: n >= 1, N > 1.\n";      // повідомлення про помилку
```

```
        return 1;                               // завершення з кодом помилки
    }

    const char* tline = "+-------+----------------------+--------+\n"; // межа трасування
    const char* thead = "| level |          n           | digit  |\n"; // шапка трасування
    int d2 = 0, d8 = 0, dN = 0;                 // глибини рекурсії

    cout << "Base 2 (recursion trace):\n" << tline << thead << tline;
    const string r2 = binary(n, 1, d2, cout);   // двійковий запис
    cout << tline;                              // нижня межа
    cout << "Base 8 (recursion trace):\n" << tline << thead << tline;
    const string r8 = octal(n, 1, d8, cout);    // вісімковий запис
    cout << tline;                              // нижня межа
    cout << "Base " << base << " (recursion trace):\n" << tline << thead << tline;
    const string rN = inBase(n, base, 1, dN, cout); // запис з основою N
    cout << tline;                              // нижня межа

    const char* line = "+--------+-----------------------------------------+-------+\n";
    cout << "Results for n = " << n << '\n' << line; // підсумкова таблиця
    cout << "|  base  | representation                          | depth |\n" << line;
    cout << "| " << setw(6) << 2 << " | " << left << setw(40) << r2 << right
         << " | " << setw(5) << d2 << " |\n";   // рядок для основи 2
    cout << "| " << setw(6) << 8 << " | " << left << setw(40) << r8 << right
         << " | " << setw(5) << d8 << " |\n";   // рядок для основи 8
    cout << "| " << setw(6) << base << " | " << left << setw(40) << rN << right
         << " | " << setw(5) << dN << " |\n";   // рядок для основи N
    cout << line;                               // нижня межа таблиці
    return 0;                                   // успішне завершення
}
#endif
```

7. Автоматизоване тестування

Файл `test_5_5.cpp`

```
// Unit-тести рекурсивних функцій переведення із завдання 5.5.
#define UNIT_TEST       // вимикаємо main() у файлі програми
#include "lab_5_5.cpp"  // підключаємо функції
#include <sstream>      // ostringstream для трасування

int main()
{
    struct Case { long long n, base; string result; int depth; }; // тестовий набір
    const Case cases[] = {{1, 2, "1", 1}, {13, 2, "1101", 4},
        {64, 8, "100", 3}, {255, 16, "FF", 2},
        {79, 40, "1[39]", 2}, {7, 100, "7", 1}};
    int failed = 0;                                 // лічильник невдалих тестів
    for (const auto& v : cases)                     // перебір наборів
    {
        int depth = 0;                              // глибина рекурсії
        ostringstream trace;                        // трасування в пам'ять
        const string actual = inBase(v.n, v.base, 1, depth, trace); // результат
        bool ok = actual == v.result && depth == v.depth;    // перевірка
        const string log = trace.str();             // текст трасування
        ok = ok && count(log.begin(), log.end(), '\n') == depth; // рядок на виклик
        if (v.base == 2)                            // окремо функція binary
        {
            depth = 0;
            ok = ok && binary(v.n, 1, depth, trace) == v.result
                    && depth == v.depth;
        }
        if (v.base == 8)                            // окремо функція octal
        {
            depth = 0;
            ok = ok && octal(v.n, 1, depth, trace) == v.result
                    && depth == v.depth;
        }
        cout << (ok ? "PASS" : "FAIL") << " n=" << v.n
             << " base=" << v.base << '\n';         // результат тесту
        if (!ok) ++failed;                          // рахуємо помилку
    }
    cout << "5.5: " << 6 - failed << "/6 passed\n";   // підсумок
    return failed ? 1 : 0;                          // код завершення для CTest
}
```

Фактичний вивід тестової програми:

```
PASS n=1 base=2
PASS n=13 base=2
PASS n=64 base=8
PASS n=255 base=16
PASS n=79 base=40
PASS n=7 base=100
5.5: 6/6 passed
```

Таблиця 3 – Результати unit-тестів

№	n	N	Очікуваний запис	Глибина	Результат
1	1	2	1	1	пройдено
2	13	2	1101	4	пройдено
3	64	8	100	3	пройдено
4	255	16	FF	2	пройдено
5	79	40	1[39]	2	пройдено
6	7	100	7	1	пройдено

8. Результати виконання програми

Компілятор clang++, стандарт C++17, ключі -Wall -Wextra -Wpedantic (попереджень немає). Вхідні дані: `13 16`.

```
n N (n >= 1, N > 1): 13 16
Base 2 (recursion trace):
+-------+----------------------+--------+
| level |          n           | digit  |
+-------+----------------------+--------+
|     1 |                   13 |      1 |
|     2 |                    6 |      0 |
|     3 |                    3 |      1 |
|     4 |                    1 |      1 |
+-------+----------------------+--------+
Base 8 (recursion trace):
+-------+----------------------+--------+
| level |          n           | digit  |
+-------+----------------------+--------+
|     1 |                   13 |      5 |
|     2 |                    1 |      1 |
+-------+----------------------+--------+
Base 16 (recursion trace):
+-------+----------------------+--------+
| level |          n           | digit  |
+-------+----------------------+--------+
|     1 |                   13 |      D |
+-------+----------------------+--------+
Results for n = 13
+--------+--------------------------------------------+-------+
|  base  | representation                             | depth |
+--------+--------------------------------------------+-------+
|      2 | 1101                                       |     4 |
|      8 | 15                                         |     2 |
|     16 | D                                          |     1 |
+--------+--------------------------------------------+-------+
```

9. Посилання на Git-репозиторій

https://github.com/dzhamsasha55-dev/lab-5

Висновки

Реалізовано рекурсивні функції зображення натурального числа у двійковій, вісімковій системах та системі з довільною основою N > 1. Цифра n mod N дописується на рекурсивному підйомі, тому цифри отримуються у правильному порядку.

Рівень кожного виклику виводиться в таблиці трасування, глибина рекурсії – у підсумковій таблиці; вона дорівнює кількості цифр результату. Усі 6 unit-тестів пройдено (перевіряються результат, глибина та кількість рядків трасування).